COMPUTER ENGINEERING

QUEZON CITY

COURSE CODE/TITLE:	CPE 010 - Data Structure and Algorithm	**SECTION:**	CPE21S2
DATE PERFORMED:	07/10/26	**DATE SUBMITTED:**	7/15/26
NAME:	Macasaet, John David C.	**INSTRUCTOR:**	Engr. Jimlord Quejado

ACTIVITY NO. ___ [ACTIVITY NAME]

1. PROCEDURE OUTPUT

ILO A:

SECTIONS	**ANSWER**
Header File Declaration Section	`#include <iostream>`

Global Declaration Section	`int sumOfTwoNum(int fNum, int sNum);`
	`bool greaterThan(int varA, int varB);`
	`bool logicalOperators(bool a, bool b);`

Class Declaration and Method Definition Section	

Main Function

```cpp
//ILO A
int main(){
    std::cout << "SUM OF TWO NUMBERS\n";
    std::cout<< "The sum of 10 and 5 is "<< sumOfTwoNum(10,5);
    std::cout << "\n------------------------\n";

    int varA, varB;
    std::cout<< "DETERMING WHICH AMONG TWO NUMBERS IS GREATER THAN THE OTHER\n";
    std::cout<< "Enter first number: ";
    std::cin >> varA;

    std::cout<< "Enter second number: ";
    std::cin >> varB;

    if (greaterThan(varA,varB)){
        std::cout << varA << " is greater than " << varB;
    }else{
        std::cout << varA << " is NOT greater than " << varB;
    }

    std::cout << "\n------------------------\n";
    bool a = true;
    bool b = false;
    std::cout <<"LOGICAL OPERATORS\n"<< "Boolean Values: TRUE, FALSE";
    if (logicalOperators(a,b)){
        std::cout << "\nThe function correctly displayed the results of the logical operators!";
    }else{
       std::cout << "\nThe function failed to display the results.";
    }

    return 0;
}
```

Method Definition

```cpp
int sumOfTwoNum(int fNum, int sNum){
    int sum = fNum + sNum;
    return sum;
} // a function was created that takes two numbers and displays the sum.
bool greaterThan(int varA, int varB){
    return varA > varB;
} // if varA is greater than varB then a boolean value of 1 will be returned.
bool logicalOperators(bool a, bool b){
   std::cout << "\nAND: " << ((a && b) ? "True" : "False");
   std::cout << "\nOR: " << ((a || b) ? "True" : "False");
   std::cout << "\nNOT A: " << ((!(a)) ? "True" : "False");
   std::cout << "\nNOT B: " << ((!(b)) ? "True" : "False");
  return true;
}// this code takes two boolean values and passes them though logical operators to see if
// it will result in True or False.
```

OUTPUT:

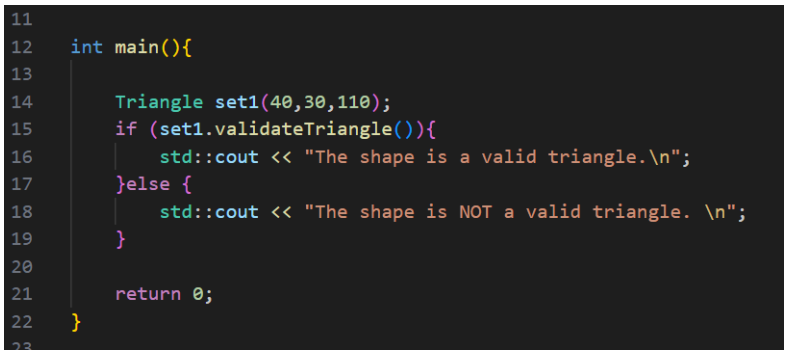
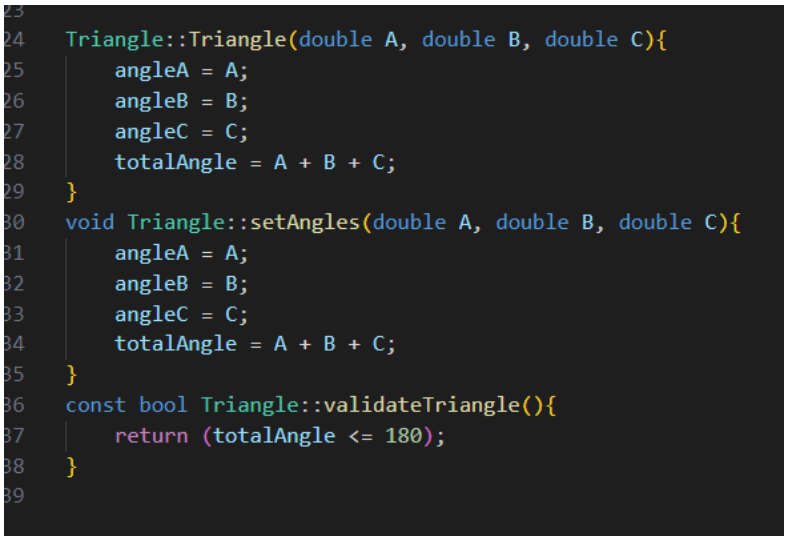
```
PROBLEMS  OUTPUT  DEBUG CONSOLE  TERMINAL  PORTS

PS C:\Users\LENOVO\Downloads\MACASAETCPE21S2\ACTIVITIES\hoa1> cd "c:\Users\LE
2\ACTIVITIES\hoa1\" ; if ($?) { g++ mainILOA.cpp -o mainILOA } ; if ($?) { .\
SUM OF TWO NUMBERS
The sum of 10 and 5 is 15
------------------------
DETERMING WHICH AMONG TWO NUMBERS IS GREATER THAN THE OTHER
Enter first number: 8
Enter second number: 9
8 is NOT greater than 9
------------------------
LOGICAL OPERATORS
Boolean Values: TRUE, FALSE
AND: False
OR: True
NOT A: False
NOT B: True
The function correctly displayed the results of the logical operators!
PS C:\Users\LENOVO\Downloads\MACASAETCPE21S2\ACTIVITIES\hoa1>
```

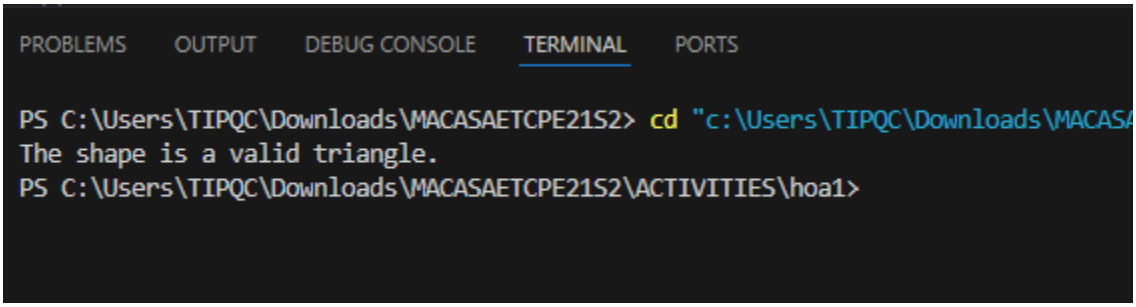
ILO B:

SECTIONS	ANSWER
Header File Declaration Section	`#include <iostream>`
Global Declaration Section	

Class Declaration and Method Definition Section	`class Triangle{`
	`private:`
	`double totalAngle, angleA, angleB, angleC;`
	`public:`
	`Triangle(double A, double B, double C);`
	`void setAngles(double A, double B, double C);`
	`const bool validateTriangle();`
	`};`

Main Function	
Method Definition	

OUTPUT:



ILO B took 3 angles and determined whether those angles formed a valid triangle. It took values and using bool if the sum of the 3 angles resulted in less than or equal to 180 then that boolean function will return a true value. This will cause a message to appear saying that the shape the angles form is a valid triangle.

Table 1-2. ILO B output observations and comments.

2. ANSWERS TO SUPPLEMENTAL ACTIVITY

ILO C:

mainILOA.cpp | mainILOC-4.cpp | mainILOC.cpp | mainILOB.cpp

ACTIVITIES > hoa1 > mainILOC.cpp > ...

```cpp
#include <iostream>
#include <cmath>
#include <iomanip>
float toFahrenheit(float kelvin);
float distance(float x1, float x2, float y1, float y2);

int main() {
    int a, b;
    std::cout<< "SWAPPING TWO NUMBERS\n";
    std::cout <<"Enter first number(a): ";
    std::cin >> a;
    std::cout << "Enter second number(b): ";
    std::cin >> b;

    int c = a;
    a = b;
    b = c;

    std::cout << "Swapped:\n" << "a = "<< a <<std::endl;
    std::cout << "b = "<< b;
    std::cout << "\n------------------\n";
    // This code swaps two values from two different variables by using a third variable
    // to store the first variable.

    std::cout <<"KELVIN TO FAHRENHEIT CONVERTER\n";
    float kelvin;
    std::cout << "Enter a Kelvin value: ";
    std::cin >> kelvin;
    std::cout<< kelvin << " Kelvin is equivalent to "<< toFahrenheit(kelvin) << " Fahrenheit";
    std::cout << "\n------------------\n";

    float x1, x2, y1, y2;
    std::cout <<"DISTANCE BETWEEN TWO POINTS\n";
    std::cout<< "Enter x1: ";
    std::cin >> x1;
    std::cout<< "Enter y1: ";
    std::cin >> y1;
    std::cout << "Enter x2: ";
    std::cin >> x2;
    std::cout <<"Enter y2: ";
    std::cin >> y2;

    std::cout << "The distance between (" << x1 << ", "<< y1<< ") and ("<< x2 <<", "<< y2<< ") is ";
    std::cout << std::setprecision(3) << distance(x1,x2,y1,y2);
    return 0;
}
float toFahrenheit(float kelvin){
    float fahrenheit = (kelvin - 273.15) *(1.8)+ 32;
  return fahrenheit;
} // This converts a kelvin value to fahrenheit using the formula above.
float distance(float x1, float x2, float y1, float y2){
    float dx = x2 - x1;
    float dy = y2 - y1;
    return sqrt((dx * dx) + (dy * dy));
} // Using the distance formula I were able to get the distance between two points.
```

```
SWAPPING TWO NUMBERS
Enter first number(a): 12
Enter second number(b): 10
Swapped:
a = 10
b = 12
------------------
KELVIN TO FAHRENHEIT CONVERTER
Enter a Kelvin value: 270
270 Kelvin is equivalent to 26.33 Fahrenheit
------------------
DISTANCE BETWEEN TWO POINTS
Enter x1: 2
Enter y1: 3
Enter x2: 5
Enter y2: 6
The distance between (2, 3) and (5, 6) is 4.24
PS C:\Users\LENOVO\Downloads\MACASAETCPE21S2\ACTIVITIES\hoa1>
```

ILO C number 4:

ACTIVITIES > hoa1 > mainILOC-4.cpp > ...

```cpp
#include <iostream>

class Triangle{
    private:
        double totalAngle, angleA, angleB, angleC, sideA, sideB, sideC, base, height;
    public:
        Triangle(double A, double B, double C);
        void setAngles(double A, double B, double C);
        const bool validateTriangle();
        double periTriangle();
        void setSides(double A, double B, double C);
        void triangleTypes();
        void setBh(double A, double B);
        double area();
};

int main(){

    Triangle set1(40,30,110);
    if (set1.validateTriangle()){
        std::cout << "The shape is a valid triangle.\n";
    }else {
        std::cout << "The shape is NOT a valid triangle. \n";
    }

    set1.triangleTypes();


    set1.setSides(10, 20, 25);
    std::cout<< "The perimeter of the triangle is " << set1.periTriangle();
    set1.setBh(20, 30);
    std::cout<< "\nThe area of the triangle is "<< set1.area();


    return 0;
}

```

```cpp
Triangle::Triangle(double A, double B, double C){
    angleA = A;
    angleB = B;
    angleC = C;
    totalAngle = A + B + C;
}
void Triangle::setAngles(double A, double B, double C){
    angleA = A;
    angleB = B;
    angleC = C;
    totalAngle = A + B + C;
}
const bool Triangle::validateTriangle(){
    return (totalAngle <= 180);
}
void Triangle::setSides(double A, double B, double C){
    sideA = A;
    sideB = B;
    sideC = C;
}
double Triangle::periTriangle(){
    double perimeter = sideA + sideB + sideC;
    return perimeter;
}

void Triangle::triangleTypes(){
    int obtuse = (angleA < 90) + (angleB < 90) + (angleC < 90);

    if (angleA < 90 && angleB < 90 && angleC < 90) {
        std::cout << "The triangle is an acute triangle\n";
    }
    else if (obtuse == 2) {
        std::cout << "The triangle is an obtuse triangle\n";
    }else{
        std::cout<<"the triange is neither obtuse or acute triangle\n";
    }
}

void Triangle::setBh(double A, double B){
    base = A;
    height = B;
}
double Triangle::area(){
    double area = (0.5)*base*height;
    return area;
} // This code added 3 new things to ILO B which is getting the area, perimeter,
// and determing whether a triangle is obtuse, acute, or other.
```

```
PROBLEMS  OUTPUT  DEBUG CONSOLE  TERMINAL  PORTS

The shape is a valid triangle.
PS C:\Users\TIPQC\Downloads\MACASAETCPE21S2\ACTIVITIES\hoa1>
                                                    > cd "c:\U
The shape is a valid triangle.
The triangle is an obtuse triangle
The perimeter of the triangle is 55
The area of the triangle is 300
PS C:\Users\TIPQC\Downloads\MACASAETCPE21S2\ACTIVITIES\hoa1>
```

Conclusion:
This activity refreshed my memory when it comes to using C++ and made me discover new things that can be done in it. Using classes and functions we were able to get the values of area, perimeter, distance, etc. These can make the code cleaner as you only need to call the function to get the result of the given values. Overall, I think I did pretty well in this HOA, I had a hard time with some of it but I managed to finish it.

3. ARTIFICIAL INTELLIGENCE (AI) USE DISCLOSURE STATEMENT

I hereby declare that artificial intelligence (AI) tools were used, if applicable, in the preparation of this academic work. The following indicates the nature and extent of AI assistance:

Tools Used:

Examples: Google Gemini, ChatGPT

Purpose of Use:

Examples: I used it to generate ideas, explain certain concepts to me, and also for code assistance.

Extent of Use:

When I faced a problem, I asked AI to give me clues on how to solve the problem and I specifically asked it to not provide me with the correct code. I also asked it for possible improvements or tweaks to make the code look cleaner.

By submitting this activity, I affirm that my use of AI complies with T.I.P.'s AI usage policy and does not exceed the permitted limits for similarity, automated content generation, or academic assistance.

I understand that any false, incomplete, or misleading disclosure may be subject to investigation in accordance with due process and may result in sanctions for violation of existing T.I.P. policies.

Lab activities, templates, and related instructional materials shall not be reuploaded, reproduced, distributed, or shared with external organizations or third parties without the prior express written permission of the Technological Institute of the Philippines.